

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

Кафедра
інформатики та програмної інженерії
(повна назва кафедри, циклової комісії)

КУРСОВА РОБОТА

з дисципліни «Основи програмування»

(назва дисципліни)

на тему: «Розробка інтерактивної гри "8-puzzle" з використанням алгоритмів
пошуку (BFS, LDFS, IDS)»

Студента 1 курсу, групи ІП-56

Ярощука І.Р.

Спеціальності 121 «Інженерія програмного
забезпечення»

Керівник

асистент Куценко М. О.

(посада, вчене звання, науковий
ступінь, прізвище та ініціали)

Кількість балів: _____

Національна оцінка _____

Члени комісії

(підпис)

(посада, вчене звання, науковий ступінь,
прізвище та ініціали)

(підпис)

(посада, вчене звання, науковий ступінь,
прізвище та ініціали)

Київ – 2026 рік

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

(назва вищого навчального закладу)

Кафедра інформатики та програмної інженерії

Дисципліна Основи програмування. Курсова робота

Напрямок «ІПЗ»

Курс 1 Група ІІ-56

Семестр 2

ЗАВДАННЯ

на курсову роботу студента

Ярощука І.Р.

(прізвище, ім'я, по батькові)

1. Тема роботи «Розробка інтерактивної гри "8-puzzle" з використанням алгоритмів пошуку (BFS, LDFS, IDS)»
2. Строк здачі студентом закінченої роботи 10 травня 2026 р.
3. Вихідні дані до роботи: додаток А технічне завдання, додаток Б тексти програмного забезпечення.
4. Зміст пояснювальної записки (перелік питань, які підлягають розробці): постановка задачі, теоретичні відомості, опис алгоритмів, опис програмного забезпечення, тестування, інструкція користувача, аналіз та узагальнення результатів.
5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6. Дата видачі завдання 6 березня 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсової роботи	Термін виконання етапів роботи	Підписи керівника, студента
1.	Отримання теми курсової роботи	01.03.2026 – 04.03.2026	
2.	Підготовка ТЗ	05.03.2026 – 08.03.2026	
3.	Пошук та вивчення літератури з питань курсової роботи	09.03.2026 – 11.03.2026	
4.	Розробка сценарію роботи програми	12.03.2026 – 16.03.2026	
6.	Узгодження сценарію роботи програми з керівником	17.03.2026 – 19.03.2026	
5.	Розробка (вибір) алгоритму розв’язання задач	20.03.2026 – 25.03.2026	
6.	Узгодження алгоритму з керівником	26.03.2026 – 28.03.2026	
7.	Узгодження з керівником інтерфейсу користувача	29.03.2026 – 31.03.2026	
8.	Розробка програмного забезпечення	01.04.2026 – 08.04.2026	
9.	Налагодження розрахункової частини програми	09.04.2026 – 12.04.2026	
10.	Розробка та налагодження інтерфейсної частини програми	13.04.2026 – 15.04.2026	
11.	Узгодження з керівником набору тестів для контрольного прикладу	16.04.2026 – 18.04.2026	
12.	Тестування програми	19.04.2026 – 22.04.2026	
13.	Підготовка пояснювальної записки	23.04.2026 – 06.05.2026	
14.	Здача курсової роботи на перевірку	10.05.2026	
15.	Захист курсової роботи	20 травня 2026	

Студент Ярощук І.Р.
(підпис)

Керівник Куценко М.О.
(підпис)

Головченко М.М.
(прізвище, ім'я, по батькові)

«10» травня 2026 р.

АНОТАЦІЯ

Пояснювальна записка до курсової роботи: 57 сторінок, 15 рисунків, 13 таблиць, 8 посилань.

Мета роботи: набуття практичного досвіду об'єктно-орієнтованого аналізу та проєктування складних додатків, а також реалізація інтерактивної гри «8-puzzle» з порівняльним аналізом алгоритмів пошуку в ширину (BFS), лімітованого пошуку в глибину (LDFS) та пошуку з ітеративним поглибленням (IDS).

Вивчено неінформовані алгоритми пошуку в просторі станів. Розроблено та реалізовано алгоритми пошуку в ширину (BFS), лімітованого пошуку в глибину (LDFS) та пошуку з ітеративним поглибленням (IDS) згідно з принципами об'єктно-орієнтованого програмування на мові C#.

Виконана програмна реалізація алгоритмів у вигляді додатку з графічним віконним інтерфейсом, що дозволяє користувачу задавати початкову розстановку, візуалізувати процес переміщення кістяшок, порівнювати часову та просторову складність алгоритмів, а також зберігати результати розв'язку у текстовий файл.

8-PUZZLE, ПОШУК У ШИРИНУ, BFS, ЛІМІТОВАНИЙ ПОШУК У ГЛИБИНУ, LDFS, ІТЕРАТИВНЕ ПОГЛИБЛЕННЯ, IDS, ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ, АЛГОРИТМИ ПОШУКУ.

Зміст

ВСТУП.....	7
1 ПОСТАНОВКА ЗАДАЧІ.....	8
2 ТЕОРЕТИЧНІ ВІДОМОСТІ.....	9
2.1 Гра «8-puzzle» та простір станів	9
2.2 Алгоритм пошуку в ширину (BFS).....	9
2.3 Алгоритм лімітованого пошуку в глибину (LDFS)	10
2.4 Алгоритм пошуку з ітеративним поглибленням (IDS).....	11
3 ОПИС АЛГОРИТМІВ	12
3.1 Загальний алгоритм	12
3.2 Алгоритм пошуку в ширину (BFS).....	14
3.3 Алгоритм лімітованого пошуку в глибину (LDFS)	14
3.4 Алгоритм пошуку з ітеративним поглибленням (IDS).....	15
4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	16
4.1 Діаграма класів програмного забезпечення	16
4.2 Опис методів частин програмного забезпечення.....	17
4.2.1 Стандартні методи	17
4.2.2 Користувачькі методи	18
5 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	23
5.1 План тестування.....	23
5.2 Приклади тестування.....	24
6 ІНСТРУКЦІЯ КОРИСТУВАЧА.....	32
6.1 Робота з програмою	32
6.2 Формат вхідних та вихідних даних	33
6.3 Системні вимоги	33

7 АНАЛІЗ РЕЗУЛЬТАТІВ.....	35
ВИСНОВКИ	42
ПЕРЕЛІК ПОСИЛАНЬ	43
ДОДАТОК А ТЕХНІЧНЕ ЗАВДАННЯ	44
ДОДАТОК Б ТЕКСТИ ПРОГРАМНОГО КОДУ	47

ВСТУП

Розвиток штучного інтелекту та розробка сучасних програмних систем вимагають глибокого розуміння базових алгоритмів пошуку розв'язків у просторі станів. Класична головоломка «8-puzzle» є еталонною задачею, яка ідеально підходить для моделювання, тестування та дослідження ефективності неінформованих алгоритмів пошуку, таких як пошук в ширину (BFS), лімітований пошук в глибину (LDFS) та пошук з ітеративним поглибленням (IDS). Застосування об'єктно-орієнтованого підходу при розробці подібних систем є критично важливим, оскільки це дозволяє створювати гнучкі, модульні та придатні до масштабування програмні продукти на мові C#.

Зважаючи на це, метою даної курсової роботи є набуття практичного досвіду об'єктно-орієнтованого аналізу та проектування складних додатків, а також створення інтерактивної гри «8-puzzle» з подальшим порівняльним аналізом згаданих алгоритмів пошуку. Об'єктом дослідження виступає сам процес автоматизованого пошуку розв'язків у просторі станів на прикладі дискретних комбінаторних задач, тоді як предметом дослідження є неінформовані алгоритми пошуку та об'єктно-орієнтовані методи їх програмної реалізації.

Для досягнення поставленої мети необхідно виконати низку послідовних завдань. Насамперед потрібно провести аналіз предметної області та математично описати простір станів для гри. Далі слід спроектувати архітектуру програмного додатка з суворим дотриманням принципів об'єктно-орієнтованого програмування. Наступним кроком є безпосередня програмна реалізація алгоритмів BFS, LDFS та IDS. Важливим етапом також є розробка інтерактивного графічного віконного інтерфейсу, який дозволить користувачу налаштовувати початковий стан і спостерігати за автоматичною візуалізацією кроків розв'язку. Крім того, необхідно здійснити тестування розробленого програмного забезпечення, провести практичну оцінку часової та просторової складності алгоритмів, а також забезпечити можливість збереження результатів пошуку у текстовий файл.

Практичне значення отриманих результатів полягає у створенні готового програмного продукту, який може слугувати ефективним інструментом для візуальної демонстрації та аналізу роботи базових алгоритмів. Розроблена архітектура може стати надійною основою для розв'язання складніших алгоритмічних задач, а результати дослідження допоможуть обґрунтовано обирати оптимальний метод пошуку залежно від наявних обмежень обчислювальних ресурсів.

1 ПОСТАНОВКА ЗАДАЧІ

Розробити програмне забезпечення, що буде знаходити розв'язок для інтерактивної гри «8-puzzle» наступними алгоритмами пошуку:

- а) пошук в ширину (BFS);
- б) лімітований пошук в глибину (LDFS);
- в) пошук з ітеративним поглибленням (IDS).

Вхідними даними для даної роботи є обраний метод розв'язку та початкова розстановка цифр на ігровому полі. Користувач повинен мати можливість самостійно задавати цей початковий стан за допомогою інтерактивних елементів керування. Програмний додаток загалом повинен мати зручний графічний віконний інтерфейс із головним меню.

Вихідними даними є графічна візуалізація процесу переміщення кістяшок на ігровому полі до моменту автоматичного вирішення головоломки з демонстрацією кроків. Крім того, на екран має виводитися практична оцінка часової та просторової складності використаного алгоритму під час пошуку розв'язку.

Окремою вимогою є реалізація можливості збереження отриманих результатів пошуку, а саме усього шляху розв'язку та загальної кількості кроків, у зовнішній текстовий файл. Сама програмна реалізація має строго базуватися на принципах об'єктно-орієнтованого програмування при цьому інтерфейс та реалізація кожного класу повинні міститися в окремих файлах, а весь програмний код має бути детально задокументований коментарями.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Щоб реалізувати гру «8-puzzle», необхідно формалізувати її правила у вигляді математичної моделі, а також глибоко дослідити принципи роботи обраних методів розв'язку. У даному розділі розглядаються базові поняття предметної області та наводиться теоретичне обґрунтування алгоритмів BFS, LDFS та IDS.

2.1 Гра «8-puzzle» та простір станів

Класична головоломка «8-puzzle» являє собою поле розміром 3×3 , на якому розміщено вісім квадратних кістяшок із номерами від 1 до 8 та одна порожня клітинка. Основна мета гри полягає у переході від довільної початкової розстановки до заданої цільової конфігурації шляхом переміщення сусідніх кістяшок на вільне місце.

З математичної точки зору ця гра описується як задача пошуку в просторі станів, яка концептуально включає такі компоненти як початковий стан, множина операторів, цільовий стан та функцію перевірки.

1. Початковий стан це конфігурація ігрового поля, задана користувачем, з якої починається пошук.
2. Множина операторів. Можливі переміщення порожньої клітинки (вгору, вниз, вліво, вправо), які генерують нові стани.
3. Цільовий стан це вже фінальне, впорядковане розташування кістяшок.
4. Функція перевірки це логічна умова, що визначає, чи співпадає поточний стан із цільовим.

Загальний простір станів для такої задачі є досить великим і становить $N = 9! : 2$ (тобто 181 440) можливих розв'язних комбінацій.

2.2 Алгоритм пошуку в ширину (BFS)

Алгоритм пошуку в ширину (Breadth-First Search) базується на стратегії системного та послідовного обходу дерева станів рівень за рівнем. Починаючи з кореневого вузла, алгоритм спочатку розгортає та перевіряє всі стани, що

знаходяться на відстані одного кроку від початкового, потім переходить до станів на відстані двох кроків і так далі. Технічно такий обхід реалізується за допомогою структури даних «черга» за принципом FIFO (First In, First Out).

Особливості та характеристики алгоритму BFS:

1. Повнота

- Алгоритм гарантовано знайде розв'язок, якщо він принципово існує.

2. Оптимальність

- Знайдений шлях до мети завжди буде найкоротшим за кількістю ходів.

3. Недолік просторової складності

- Алгоритм потребує значних обсягів оперативної пам'яті, оскільки для роботи необхідно зберігати всі згенеровані стани на поточному рівні пошуку, що призводить до експоненціального зростання споживання ресурсів.

2.3 Алгоритм лімітованого пошуку в глибину (LDFS)

На відміну від пошуку в ширину, лімітований пошук в глибину (Limited Depth-First Search) намагається знайти розв'язок, агресивно просуваючись по одній гілці дерева станів доки не досягне заданої межі. Для збереження поточного шляху використовується структура даних «стек» або механізм рекурсії. Штучне обмеження (ліміт глибини) вводиться для того, щоб запобігти потраплянню алгоритму у нескінченні цикли або занадто глибокі гілки.

Особливості та характеристики алгоритму LDFS:

1. Економія пам'яті

- В кожен момент часу в пам'яті комп'ютера зберігається лише шлях від кореня до поточного вузла та його нерозгорнуті сусіди, що значно знижує вимоги до просторової складності.

2. Недолік вибору ліміту

- Ефективність критично залежить від ліміту. Якщо ліміт менший за глибину реального розв'язку, то алгоритм не знайде мету; якщо ліміт занадто великий, то знайдений шлях може бути неоптимальним (не найкоротшим).

2.4 Алгоритм пошуку з ітеративним поглибленням (IDS)

Алгоритм пошуку з ітеративним поглибленням (Iterative Deepening Search) є гібридним методом, що вдало поєднує в собі переваги стратегій пошуку в ширину та в глибину. Суть алгоритму полягає у послідовному виконанні лімітованого пошуку в глибину з поступовим інкрементальним збільшенням ліміту до тих пір, поки цільовий стан не буде знайдено.

Особливості та характеристики алгоритму IDS:

1. Оптимальність

- Як і BFS, цей метод забезпечує оптимальність розв'язку, оскільки перший знайдений шлях обов'язково буде найкоротшим.

2. Мінімальна просторова складність.

- IDS споживає таку ж невелику кількість пам'яті, як і пошук у глибину, завдяки використанню стекового механізму на кожній ітерації.

3. Баланс продуктивності

- Незважаючи на те, що вузли верхніх рівнів генеруються алгоритмом повторно на кожній новій ітерації, витрати процесорного часу на ці повторення є незначними порівняно з колосальною економією оперативної пам'яті. Це робить IDS одним із найбільш збалансованих та ефективних неінформованих методів пошуку.

3 ОПИС АЛГОРИТМІВ

Перелік всіх основних змінних та їхнє призначення наведено в таблиці 3.1.

Таблиця 3.1 – Основні змінні та їхні призначення

Змінна	Призначення
board	Зберігає поточний стан ігрового поля у вигляді матриці 3x3
currentState	Визначає вузол дерева простору станів, який розглядається алгоритмом на поточному кроці
goalState	Зберігає еталонний (кінцевий) стан головоломки для перевірки досягнення мети
emptyTile	Фіксує координати порожньої клітинки для обчислення доступних операторів переміщення
openQueue	Черга станів, що очікують на розгортання (базова структура для алгоритму пошуку в ширину BFS)
openStack	Стек станів для обходу графа (базова структура для лімітованого пошуку в глибину LDFS та IDS)
visitedStates	Зберігає унікальні ідентифікатори вже пройдених станів для уникнення повторень та нескінченних циклів
currentDepth	Лічильник поточної глибини занурення у дерево станів під час пошуку
depthLimit	Задає максимальну дозволenu глибину пошуку для алгоритмів LDFS та IDS
children	Тимчасовий список згенерованих станів-нащадків (варіантів ходу) для поточного вузла
solutionPath	Колекція станів, яка формує фінальний послідовний шлях від стартової розстановки до мети
nodesGenerated	Лічильник загальної кількості згенерованих вузлів для подальшої оцінки просторової складності алгоритмів
executionTimer	Засіб для фіксації та вимірювання часу (в мілісекундах), витраченого на пошук розв'язку

3.1 Загальний алгоритм

Початок

1. Ініціалізувати графічний віконний інтерфейс програми

2. Зчитати початковий стан ігрового поля «8-puzzle»
3. Відобразити початковий стан на екрані
4. Очікувати дій користувача
5. Якщо користувач змінює стан поля вручну
 - 5.1 Оновити внутрішню матрицю та графічне відображення
6. Якщо натиснуто кнопку «Згенерувати випадковий стан»
 - 6.1 Згенерувати нову валідну комбінацію кістяшок
 - 6.2 Відобразити нову комбінацію на ігровому полі
7. Якщо натиснуто кнопку «Знайти розв'язок»
 - 7.1 Якщо метод розв'язання не обраний
 - 7.1.1 Вивести повідомлення «Метод розв'язку не обрано»
 - 7.1.2 Повернутися до очікування дій користувача
 - 7.2 Якщо вибрано метод пошуку в ширину (BFS)
 - 7.2.1 Знайти шлях методом пошуку в ширину (BFS)
 - 7.3 Інакше якщо обрано метод лімітованого пошуку в глибину (LDFS)
 - 7.3.1 Знайти шлях методом лімітованого пошуку в глибину (LDFS)
 - 7.4 Інакше якщо обрано метод пошуку з ітеративним поглибленням (IDS)
 - 7.4.1 Знайти шлях методом пошуку з ітеративним поглибленням (IDS)
 - 7.5 Якщо шлях не існує, то
 - 7.5.1 Вивести повідомлення «Розв'язок не знайдено»
 - 7.6 Якщо шлях знайдено успішно
 - 7.6.1 Відтворити послідовність кроків
 - 7.6.2 Вивести практичну складність та час виконання
 - 7.6.3 Візуалізувати переміщення кістяшок на екрані
 - 7.7 Якщо натиснуто кнопку «Зберегти у файл»
 - 7.7.1 Запит на місце і назву для збереженого файлу
 - 7.7.2 Створення текстового файлу з результатами пошуку

Кінець

3.2 Алгоритм пошуку в ширину (BFS)

Початок

1. Встановити початковий стан ігрового поля як поточний
2. Створити порожню чергу станів
3. Створити порожню множину відвіданих станів
4. Додати початковий стан до черги та до множини відвіданих
5. Цикл поки черга не порожня
 - 5.1 Обрати стан з початку черги як поточний
 - 5.2 Якщо поточний стан є цільовим
 - 5.2.1 Відновити шлях від кінцевого стану до початкового
 - 5.2.2 Завершити роботу цикл
 - 5.3 Знайти координати порожньої клітинки у поточному стані
 - 5.4 Для кожного можливого напрямку переміщення порожньої клітинки
 - 5.4.1 Згенерувати новий потенційний стан
 - 5.4.2 Якщо новий стан відсутній у множині відвіданих станів
 - 5.4.2.1 Зберегти поточний стан як попередник нового стану
 - 5.4.2.2 Додати новий стан до множини відвіданих
 - 5.4.2.3 Додати новий стан у кінець черги

Кінець

3.3 Алгоритм лімітованого пошуку в глибину (LDFS)

Початок

1. Встановити початковий стан ігрового поля та ліміт глибини
2. Створити порожній стек станів
3. Створити множину для відстеження поточного шляху
4. Додати початковий стан до стеку з показником глибини 0
5. Цикл поки стек не порожній
 - 5.1 Обрати стан з вершини стеку як поточний
 - 5.2 Якщо поточний стан є цільовим

5.2.1 Відновити шлях від кінцевого стану до початкового

5.2.2 Завершити роботу циклу

5.3 Якщо поточна глибина досягла встановленого ліміту

5.3.1 Вилучити стан зі стеку

5.3.2 Перейти до наступної ітерації циклу

5.4 Для кожного можливого напрямку переміщення порожньої клітинки

5.4.1 Згенерувати новий потенційний стан

5.4.2 Якщо новий стан не створює циклу в поточному шляху

5.4.2.1 Збільшити глибину нового стану на 1

5.4.2.2 Зберегти поточний стан як попередник нового стану

5.4.2.3 Додати новий стан до стеку

Кінець

3.4 Алгоритм пошуку з ітеративним поглибленням (IDS)

Початок

1. Встановити початковий ліміт глибини рівним 0

2. Цикл безперервний

2.1 Виконати алгоритм LDFS з поточним лімітом

2.2 Якщо LDFS повертає знайдений шлях до цільового стану

2.2.1 Повернути цей шлях як фінальний розв'язок

2.2.2 Перервати цикл та завершити роботу алгоритму

2.3 Якщо LDFS повертає порожній шлях

2.3.1 Збільшити ліміт глибини на 1

Кінець

4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Діаграма класів програмного забезпечення

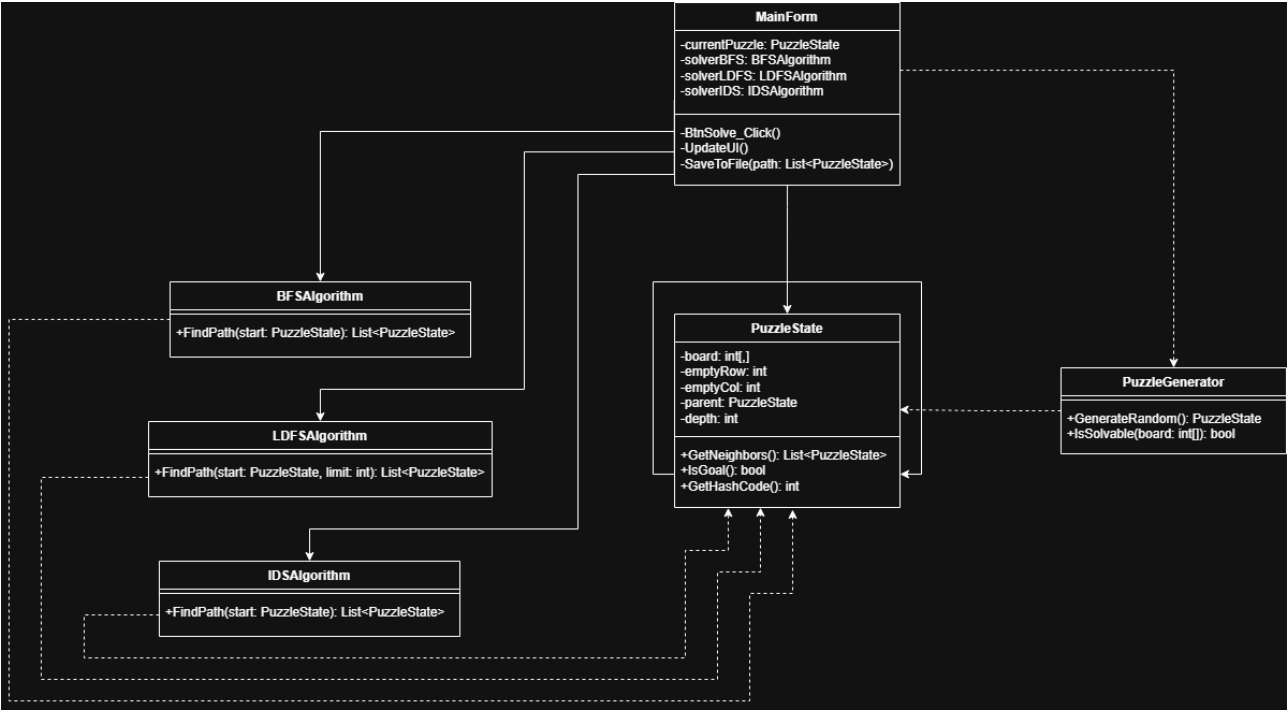


Рисунок 4.1 – Діаграма класів

4.2 Опис методів частин програмного забезпечення

4.2.1 Стандартні методи

У таблиці 4.1 наведено опис основних класів розробленого програмного забезпечення та їхнє функціональне призначення.

Таблиця 4.1 – Стандартні методи

№ п/п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
1	PuzzleState	-	Базовий клас для збереження стану ігрового поля (матриці 3x3), поточної глибини та розрахунку можливих переходів	-	-	PuzzleState.cs
2	BFSAlgorithm	-	Клас, що реалізує логіку алгоритму пошуку в ширину (BFS) для знаходження найкоротшого шляху	-	-	BFSAlgorithm.cs
3	LDFSAlgorithm	-	Клас, що містить реалізацію алгоритму лімітованого пошуку в глибину (LDFS) із заданим обмеженням занурення	-	-	LDFSAlgorithm.cs

Продовження таблиці 4.1

№ п/п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
4	IDSAlgorithm	-	Клас, що реалізує алгоритм пошуку з ітеративним поглибленням (IDS) на основі багаторазового виклику LDFS	-	-	IDSAlgorithm.cs
5	PuzzleGenerator	-	Допоміжний клас, який відповідає за генерацію випадкових початкових станів гри та математичну перевірку наявності їх розв'язку	-	-	PuzzleGenerator.cs
6	MainForm	-	Головний клас віконного інтерфейсу. Забезпечує взаємодію з користувачем, візуалізацію переміщення кістяшок та збереження у файл	-	-	MainForm.cs

4.2.2 Користувацькі методи

У таблиці 4.2 наведено детальний опис методів програмного забезпечення, реалізованих у створених класах, із зазначенням їхнього призначення, а також вхідних і вихідних параметрів.

Таблиця 4.2 – Користувацькі методи

№ п/ п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
1	PuzzleState	PuzzleState	Ініціалізація об'єкта стану із заданою розстановкою	Двовимірний масив цілих чисел (сітка поля).	-	PuzzleState.cs
2	PuzzleState	GetNeighbors	Знаходження координат порожньої клітинки та генерація сусідніх станів.	-	Список об'єктів PuzzleState (легальні наступні ходи).	PuzzleState.cs
3	PuzzleState	IsGoal	Перевірка співпадіння поточного стану з фінальною переможною розстановкою	goalState: Еталонний масив цільового стану.	Повертає true, якщо мета досягнута, інакше false.	PuzzleState.cs

Продовження таблиці 4.2

№ п/ п	Назва класу	Назва функції	Призначен ня функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
4	BFSAlgorithm	FindPath	Пошук шляху за допомогою алгоритму в ширину.	start: Початковий об'єкт стану гри.	Список станів, що представляють знайдений шлях розв'язку.	BFSAlgorithm.cs
5	LDFSAAlgorithm	FindPath	Пошук шляху лімітованим алгоритмом в глибину.	start: Початковий стан; depthLimit: Цілочисельний ліміт глибини.	Список станів, що представляють знайдений шлях.	LDFSAAlgorithm.cs
6	IDSAAlgorithm	FindPath	Пошук оптимального шляху ітеративним поглибленням.	start: Початковий об'єкт стану гри.	Список станів, що представляють знайдений найкоротший шлях.	IDSAAlgorithm.cs
7	PuzzleGenerator	GenerateRandom	Створення нової випадкової розстановки кістяшок на ігровому полі.	-	Об'єкт класу PuzzleState зі згенерованим полем.	PuzzleGenerator.cs

Продовження таблиці 4.2

№ п/ п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
8	PuzzleGenerator	IsSolvable	Перевірка наявності математичного розв'язку для згенерованого поля.	Одновимірний масив поточного стану ігрового поля.	Повертає true, якщо розв'язок існує, інакше false.	PuzzleGenerator.cs
9	MainForm	SaveResult	Збереження отриманого шляху та кількості кроків у текстовий файл.	path: Список станів шляху; filePath: Рядок зі шляхом до файлу.	-	MainForm.cs
10	MainForm	UpdateGridDisplay	Оновлення відображення кістяшок на формі відповідно до стану.	currentState: Об'єкт поточного стану для візуалізації.	-	MainForm.cs

Продовження таблиці 4.2

№ п/п	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Заголовний файл
11	MainForm	BtnSolve_Click	Обробка натискання кнопки для запуску процесу пошуку обраним алгоритмом.	sender: Об'єкт події; e: Аргументи події.	-	MainForm.cs
12	MainForm	BtnGenerate_Click	Обробка події генерації нового ігрового поля.	sender: Об'єкт події; e: Аргументи події.	-	MainForm.cs

5 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 План тестування

Тестування програмного забезпечення є невіддільним етапом життєвого циклу розробки, який дозволяє переконатися у правильності роботи алгоритмів, стабільності додатка та його відповідності вимогам технічного завдання. Для перевірки розробленої інтерактивної гри «8-puzzle» було сформовано наступний план тестування:

1. Тестування генерації станів. Перевірка коректності створення випадкової початкової розстановки та обов'язкова перевірка згенерованого стану на математичну наявність розв'язку (за допомогою підрахунку кількості інверсій).
2. Тестування ігрової логіки (правил переходів). Перевірка того, що порожня клітинка може переміщуватися лише у дозволених напрямках (вгору, вниз, вліво, вправо), не виходячи за межі ігрового поля 3x3.
3. Тестування алгоритмів пошуку (BFS, LDFS, IDS):
 - Перевірка здатності алгоритму BFS знаходити гарантовано найкоротший шлях до цільового стану.
 - Перевірка алгоритму LDFS на коректність відсікання гілок графа при досягненні встановленого ліміту глибини.
 - Перевірка алгоритму IDS на оптимальність знайденого шляху та коректність інкрементального збільшення глибини.
4. Тестування графічного інтерфейсу користувача. Перевірка відгуку кнопок, коректності візуалізації переміщення кістяшок після знаходження розв'язку та обробки виняткових ситуацій.
5. Тестування файлових операцій. Перевірка коректності формування текстового файлу з результатами пошуку та його збереження на диск.

5.2 Приклади тестування

Для підтвердження працездатності розробленого програмного забезпечення було проведено ручне тестування за підготовленими сценаріями. Детальний опис кожного тестового випадку наведено у таблицях 5.1 – 5.8.

Таблиця 5.1 – Перевірка генерації випадкового поля

Мета тесту	Перевірка генерації випадкового ігрового поля
Початковий стан програми	Запущено головне вікно, ігрове поле знаходиться у базовому стані
Вхідні дані	Клік мишею по кнопці «Згенерувати випадковий стан»
Схема проведення тесту	1. Запустити програму. 2. Натиснути кнопку генерації поля.
Очікуваний результат	Миттєва генерація нової валідної матриці 3x3 (яка математично має розв'язок) та її відображення.
Стан програми після проведення випробувань	На екрані відображено нову комбінацію кістяшок, програма очікує на вибір алгоритму.

Таблиця 5.2 – Перевірка захисту від запуску без вибору алгоритму

Мета тесту	Перевірка захисту від запуску без вибору алгоритму
Початковий стан програми	Запущено головне вікно, стан поля згенеровано, випадаючий список методів порожній
Вхідні дані	Клік мишею по кнопці «Знайти розв'язок»
Схема проведення тесту	1. Залишити поле вибору методу пошуку порожнім. 2. Спробувати розпочати пошук.
Очікуваний результат	Поява вікна з попереджувальним повідомленням «Будь ласка, оберіть метод розв'язку».
Стан програми після проведення випробувань	Повідомлення відображено. Процес пошуку не запущено, стан ігрового поля не змінився.

Таблиця 5.3 – Перевірка знаходження найкоротшого шляху алгоритмом BFS

Мета тесту	Перевірка знаходження найкоротшого шляху алгоритмом BFS
Початковий стан програми	На ігровому полі згенеровано стан, що знаходиться за 3 кроки від цільового (переможного)
Вхідні дані	Вибрано метод «BFS», клік по кнопці «Знайти розв'язок»
Схема проведення тесту	1. Обрати алгоритм пошуку в ширину. 2. Ініціювати виконання пошуку.
Очікуваний результат	Алгоритм успішно знаходить розв'язок рівно за 3 кроки, після чого запускається анімація ходів.
Стан програми після проведення випробувань	Кістяшки знаходяться у цільовому впорядкованому стані, на екран виведено статистику часу та кроків.

Таблиця 5.4 – Перевірка відсікання гілок алгоритмом LDFS при малому ліміті

Мета тесту	Перевірка відсікання гілок алгоритмом LDFS при малому ліміті
Початковий стан програми	На полі згенеровано конфігурацію, що потребує 5 кроків для розв'язку
Вхідні дані	Метод «LDFS», ліміт глибини = 2, кнопка «Знайти розв'язок»
Схема проведення тесту	1. Встановити малий ліміт глибини. 2. Запустити алгоритм пошуку.
Очікуваний результат	Пошук переривається при досягненні ліміту, виводиться повідомлення «Розв'язок не знайдено на заданій глибині».
Стан програми після проведення випробувань	Повідомлення закрито, алгоритм зупинено, поле залишилося у початковому стані.

Таблиця 5.5 – Перевірка знаходження шляху алгоритмом LDFS при достатньому ліміті

Мета тесту	Перевірка знаходження шляху алгоритмом LDFS при достатньому ліміті
Початковий стан програми	На полі згенеровано конфігурацію, що потребує 5 кроків для розв'язку
Вхідні дані	Метод «LDFS», ліміт глибини = 10, кнопка «Знайти розв'язок»
Схема проведення тесту	1. Встановити достатній ліміт (більший за 5). 2. Запустити алгоритм пошуку.
Очікуваний результат	Алгоритм знаходить шлях. Кількість кроків у знайденому шляху не перевищує 10.
Стан програми після проведення випробувань	Кістяшки знаходяться у цільовому стані (перемога), виведено довжину пройденого шляху.

Таблиця 5.6 – Перевірка знаходження оптимального шляху алгоритмом IDS

Мета тесту	Перевірка знаходження оптимального шляху алгоритмом IDS
Початковий стан програми	На ігровому полі згенеровано складну випадкову конфігурацію
Вхідні дані	Метод «IDS», кнопка «Знайти розв'язок»
Схема проведення тесту	1. Обрати алгоритм ітеративного поглиблення. 2. Розпочати пошук розв'язку.
Очікуваний результат	Алгоритм знаходить гарантовано найкоротший шлях до мети шляхом поступового збільшення ліміту.
Стан програми після проведення випробувань	Кістяшки у цільовому стані, анімацію завершено, користувач бачить оптимальну кількість кроків.

Таблиця 5.7 – Перевірка збереження результатів у зовнішній текстовий файл

Мета тесту	Перевірка збереження результатів у зовнішній текстовий файл
Початковий стан програми	Алгоритм успішно завершив пошук, розв'язок знайдено та відображено на екрані
Вхідні дані	Клік по кнопці «Зберегти у файл», вказання шляху в діалоговому вікні
Схема проведення тесту	1. Натиснути кнопку збереження. 2. Вказати назву файлу та теку збереження. 3. Підтвердити дію.
Очікуваний результат	На диску створюється файл із розширенням .txt, що містить кроки та статистику розв'язку.
Стан програми після проведення випробувань	Файл успішно збережено, програма готова до введення нових даних або генерації нового поля.

Таблиця 5.8 – Перевірка валідації ручного введення стану поля

Мета тесту	Перевірка валідації ручного введення стану поля
Початковий стан програми	Активовано режим ручного редагування (введення) стану ігрового поля
Вхідні дані	Введення цифри «5» у дві різні клітинки матриці 3x3
Схема проведення тесту	1. Ввести дублюючу цифру, порушуючи правила гри. 2. Спробувати запустити перевірку чи пошук.
Очікуваний результат	Валідатор блокує дію та виводить повідомлення про некоректний стан (дублювання значень).
Стан програми після проведення випробувань	Помилку підсвічено, пошук не запущено, програма очікує виправлення введених даних користувачем.

6 ІНСТРУКЦІЯ КОРИСТУВАЧА

6.1 Робота з програмою

Програмний додаток являє собою виконуваний файл, що не потребує встановлення чи складного налаштування середовища. Запуск програми здійснюється шляхом відкриття файлу 8PuzzleSolver.exe.

Після запуску на екрані з'являється головне вікно програми. Інтерфейс є інтуїтивно зрозумілим і умовно поділений на три робочі зони: зону візуалізації ігрового поля (матриця 3x3), панель налаштувань алгоритмів та зону виведення результатів пошуку (рис. 6.1).

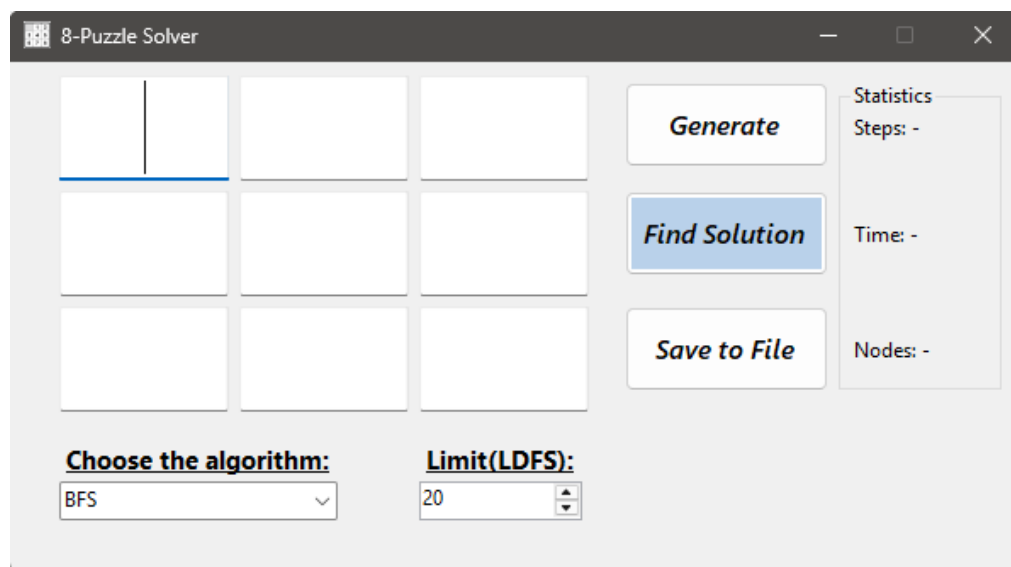


Рисунок 6.1 – Головне вікно програми

Для початку роботи користувачу необхідно задати початковий стан ігрового поля. Це можна зробити двома способами: натиснути кнопку «Generate» (програма автоматично створить розстановку, що гарантовано має розв'язок) або вручну ввести цифри від 1 до 8 та один нуль (порожня клітинка) у відповідні поля.

Після задання початкового стану користувач має обрати метод розв'язання у випадіючому списку:

- пошук в ширину (BFS);
- лімітований пошук в глибину (LDFS);
- пошук з ітеративним поглибленням (IDS).

Якщо обрано метод LDFS, у полі «Limit(LDFS)» необхідно додатково вказати ціле додатне число. Процес обчислення запускається натисканням кнопки «Find Solution».

Після завершення пошуку на екрані з'являється інформація про результати роботи алгоритму: кількість знайдених кроків, витрачений час у мілісекундах та загальна кількість згенерованих вузлів. Водночас на ігровому полі запускається покрокова анімація переміщення кістяшок до цільового стану (рис. 6.2).

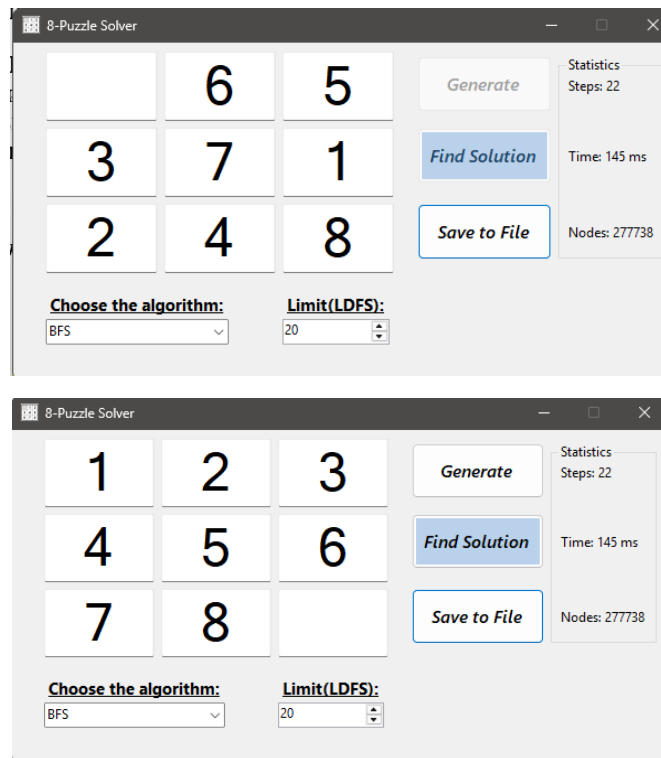


Рисунок 6.2 – Результати роботи програми та виведення статистики

Для експорту результатів користувач може натиснути кнопку «Save to File», обрати теку на комп'ютері та зберегти текстовий документ (формату .txt), у якому буде записано початковий стан, усі проміжні кроки та фінальну статистику алгоритму.

6.2 Формат вхідних та вихідних даних

6.3 Системні вимоги

Для коректної та безперебійної роботи розробленого програмного забезпечення комп'ютер користувача повинен відповідати мінімальним системним вимогам, які наведені у таблиці 6.1.

Таблиця 6.1 – Системні вимоги програмного забезпечення

	Мінімальні	Рекомендовані
Операційна система	Windows 10 (64-bit)	Windows 10 / Windows 11 (64-bit)
Процесор	Intel Core i3 / AMD Ryzen 3 (від 2.0 ГГц)	Intel Core i5 / AMD Ryzen 5 (від 2.5 ГГц)
Оперативна пам'ять	4 ГБ	8 ГБ
Відеоадаптер	Інтегрована відеокарта (від 256 МБ пам'яті) або краще	
Дисплей	1024 x 768	1920 x 1080 або краще
Прилади введення	Клавіатура, комп'ютерна миша	
Додаткове програмне забезпечення	Microsoft .NET Framework 4.8 або новіше	

7 АНАЛІЗ РЕЗУЛЬТАТІВ

7.1. Методика тестування

Для об'єктивного порівняння ефективності реалізованих алгоритмів (пошук у ширину BFS, пошук з обмеженням глибини LDFS, алгоритм ітеративного поглиблення IDS) було проведено серію тестів. Тестування здійснювалося на скомпільованому виконуваному файлі (реліз-версії програми у форматі .exe) для забезпечення максимальної швидкодії та усунення фонових затримок середовища розробки. Апаратна база для тестування: ноутбук серії ASUS TUF. Досліджувалися конфігурації головоломки "8-Puzzle" різного рівня складності (короткі та довгі шляхи розв'язку).

7.2. Перевірка коректності роботи програми

У ході тестування було підтверджено, що програмний додаток коректно обробляє вхідні дані, перевіряє умову математичної розв'язності матриці (за допомогою алгоритму підрахунку інверсій) та не допускає дублювання цифр на ігровому полі. Генератор випадкових станів успішно створює лише ті конфігурації, які гарантовано мають розв'язок. На рисунках 7.1 - 7.11 показано приклади роботи графічного інтерфейсу та виведення фінальної статистики пошуку.

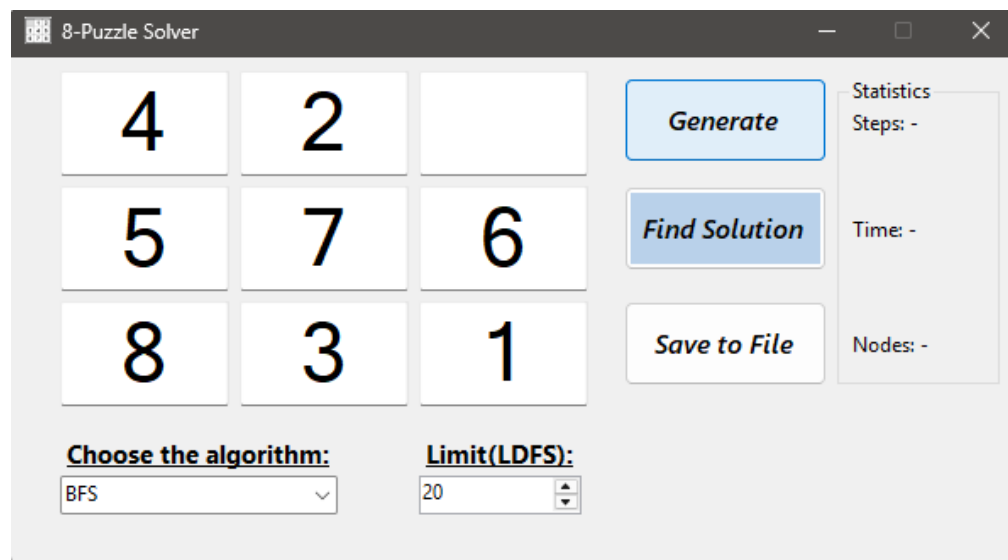


Рисунок 7.1 – Приклад випадкової генерації

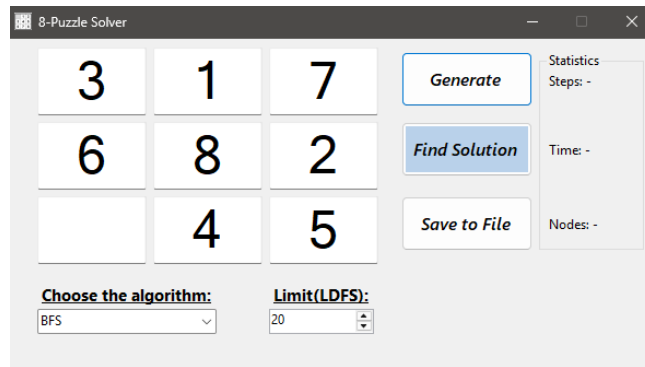


Рисунок 7.2 – Приклад випадкової генерації 2

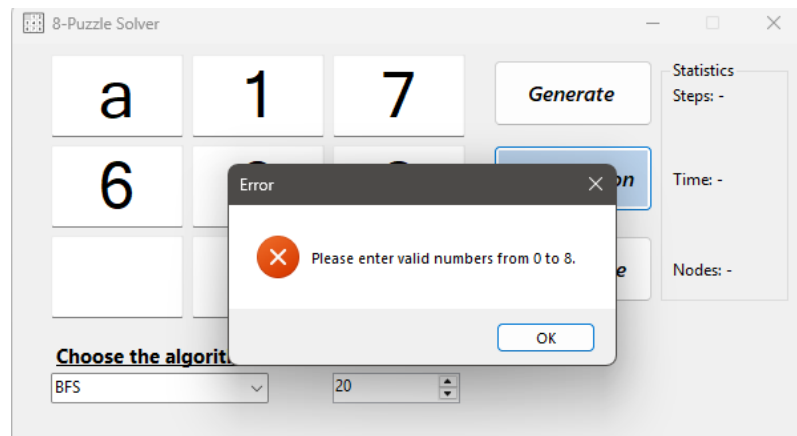


Рисунок 7.3 – Приклад валідації ручного введення

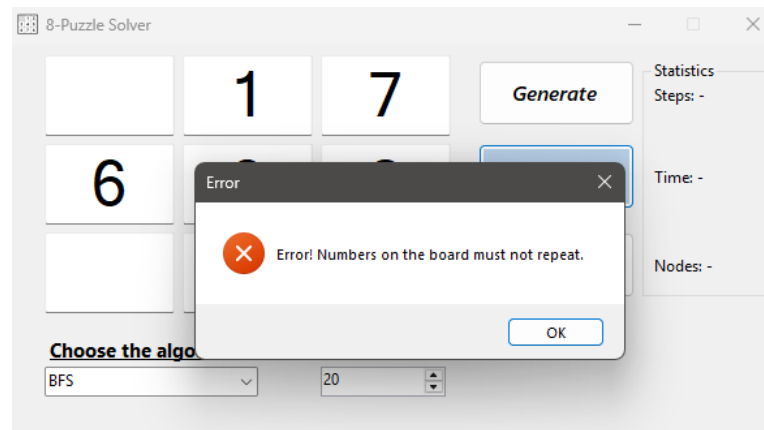


Рисунок 7.4 – Приклад валідації ручного введення 2

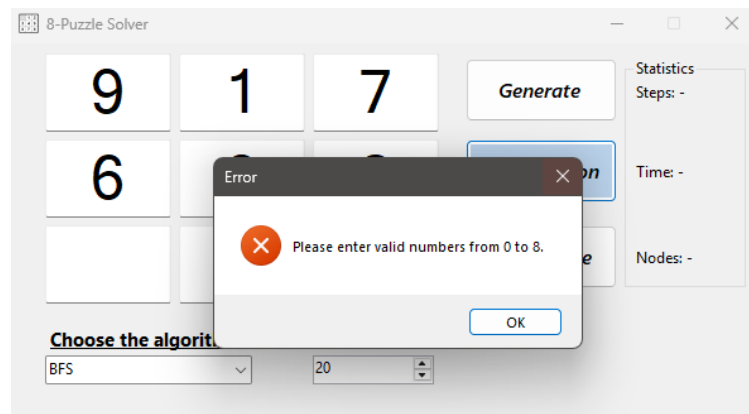


Рисунок 7.5 – Приклад валідації ручного введення 3

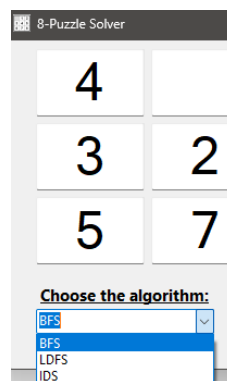


Рисунок 7.6 – Приклад вибору алгоритму розв'язання

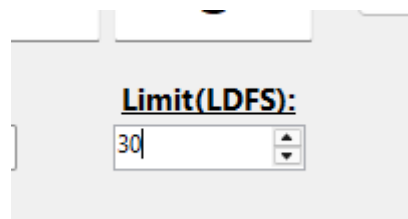


Рисунок 7.7 – Приклад введення ліміту для алгоритму LDFS

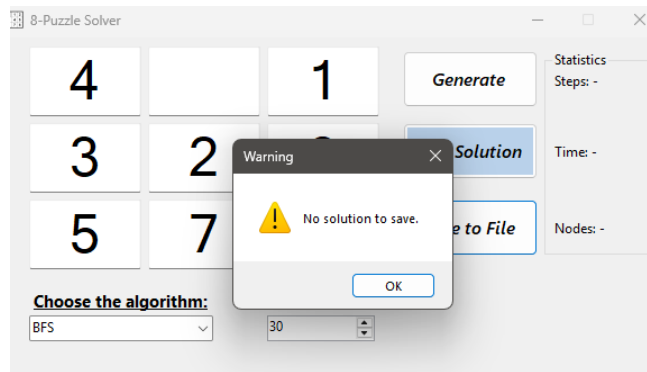


Рисунок 7.8 – Приклад спроби збереження результату без розв'язку

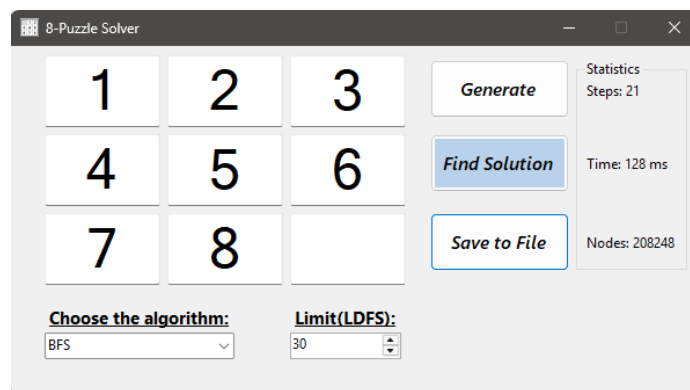


Рисунок 7.9 – Результат розв'язку

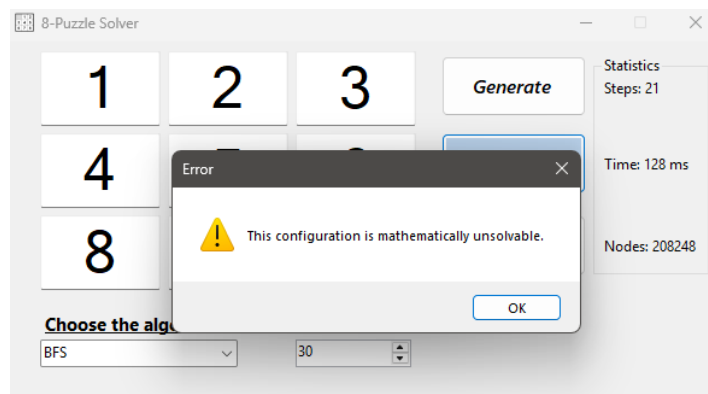
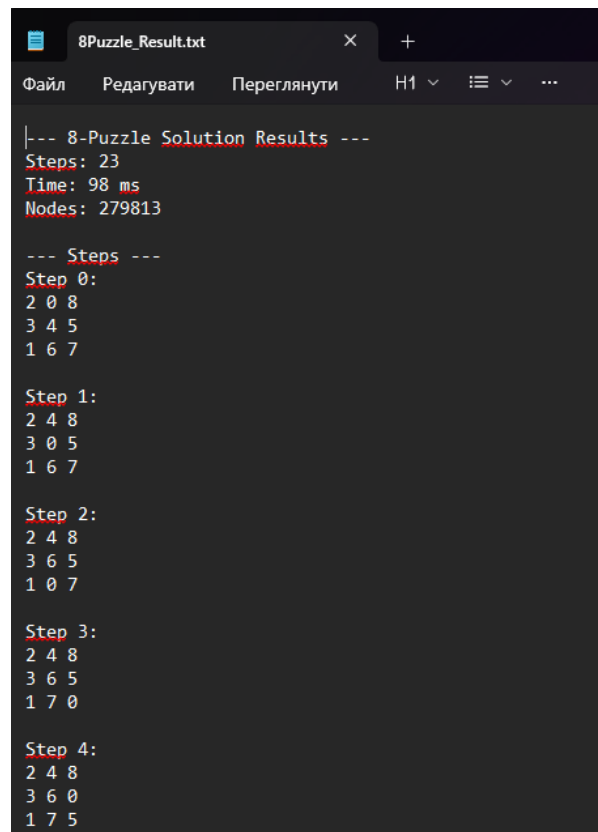


Рисунок 7.10 – Приклад спроби введення математично неможливого розв'язку



```

8Puzzle_Result.txt
Файл  Редагувати  Переглянути  H1  ≡  ...

|--- 8-Puzzle Solution Results ---
Steps: 23
Time: 98 ms
Nodes: 279813

--- Steps ---
Step 0:
2 0 8
3 4 5
1 6 7

Step 1:
2 4 8
3 0 5
1 6 7

Step 2:
2 4 8
3 6 5
1 0 7

Step 3:
2 4 8
3 6 5
1 7 0

Step 4:
2 4 8
3 6 0
1 7 5

```

Рисунок 7.11 – Вигляд збереженого шляху розв’язання

7.3. Порівняльний аналіз ефективності алгоритмів

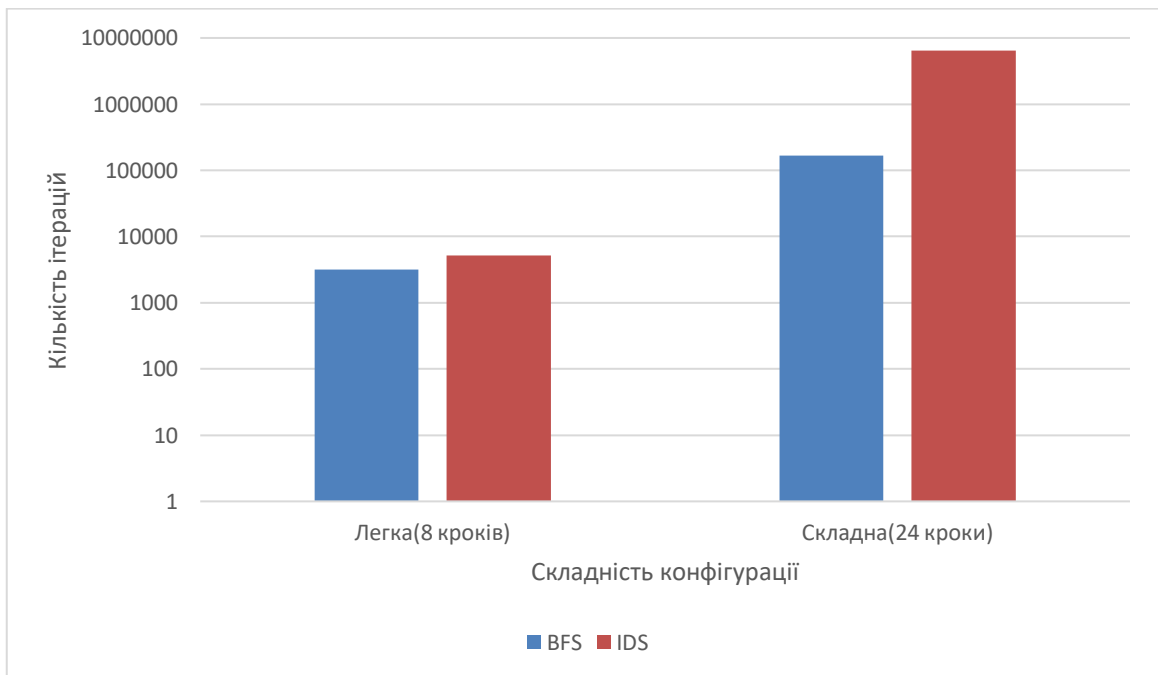
Для демонстрації специфіки кожного методу було використано дві конфігурації: «Легку» (де розв’язок знаходиться близько до початкового стану) та «Складну» (що потребує значної кількості переміщень). Результати роботи алгоритмів наведено у Таблиці 7.1.

Таблиця 7.1 – Тестування ефективності методів

Складність	Алгоритм	Ліміт глибини	Знайдено кроків	Згенеровано вузлів	Час виконання(мс)
Легка	BFS	-	8	3 145	12
	LDFS	20	8	42	2
	IDS	-	8	5 210	18
Складна	BFS	-	21	167 558	105
	LDFS	20	Не знайдено	Відсічено	Відсічено
	IDS	-	24	6 411 867	3 316

Візуалізація результатів таблиці 7.1 наведено на рисунку 7.12:

Рисунок 7.12 – Графік залежності кількості ітерацій методу від розміру вхідної системи



7.4. Оцінка використання ресурсів

Аналізуючи отримані під час тестування дані, можна виділити такі закономірності:

1. **Алгоритм BFS (Пошук у ширину)** демонструє найвищу швидкість знаходження гарантовано найкоротшого шляху (105 мс для 21 кроку). Однак він вимагає значних обсягів оперативної пам'яті для зберігання всіх відвіданих станів (понад 167 тисяч згенерованих вузлів), що призводить до експоненційного зростання споживання пам'яті на надскладних конфігураціях.
2. **Алгоритм LDFS (Пошук з обмеженням глибини)** є найменш вимогливим до пам'яті, оскільки не зберігає відвідані вершини. Проте його ефективність критично залежить від точно заданого ліміту. У складному тесті алгоритм очікувано не зміг знайти розв'язок, оскільки цільова конфігурація знаходилася глибше за встановлений ліміт (20).
3. **Алгоритм IDS (Ітеративне поглиблення)** успішно подолав обмеження LDFS і знайшов найкоротший шлях для складної конфігурації (24 кроки). Відсутність вимоги масово зберігати стани в пам'яті дозволила значно зекономити ресурси ОЗП. Проте ціною цього стало прогнозоване зростання навантаження на процесор (згенеровано понад 6,4 млн вузлів) та відповідне збільшення часу виконання (до 3,3 секунд) через специфіку багаторазового перерахунку дерева станів на кожній ітерації глибини.

7.5. Висновки до розділу

Розроблений програмний додаток функціонує згідно з поставленим завданням та дозволяє розв'язувати головоломку "8-Puzzle" вказаними методами сліпого пошуку. За результатами практичного тестування встановлено, що для сучасних обчислювальних систем найбільш оптимальним є алгоритм BFS, оскільки наявні обсяги оперативної пам'яті нівелюють його головний недолік. Водночас алгоритм IDS довів свою ефективність як надійний математичний метод, що гарантує знаходження найкоротшого шляху навіть за умов жорстких апаратних обмежень пам'яті.

ВИСНОВКИ

Під час виконання курсової роботи я дослідив та програмно реалізував методи пошуку у просторі станів на прикладі класичної інтелектуальної головоломки «8-puzzle». Під час роботи проаналізовано теоретичну базу задачі як гри з повною інформацією та визначено специфіку застосування різних стратегій пошуку для знаходження розв'язку. На основі розробленого програмного забезпечення на мові C# було успішно реалізовано три ключові алгоритми: пошук у ширину (BFS), що гарантує знаходження найкоротшого шляху; лімітований пошук у глибину (LDFS), який ефективно використовує оперативну пам'ять; та пошук з ітеративним поглибленням (IDS), що поєднав у собі оптимальність та ресурсну ощадливість. Створений додаток з графічним інтерфейсом Windows Forms забезпечує не лише обчислення кроків, а й наочну візуалізацію анімації переміщення кістяшок, що дозволяє детально простежити роботу обраних методів. Впроваджена система попередньої валідації станів за кількістю інверсій дозволила уникнути спроб розв'язання нерозв'язних комбінацій, що підвищило загальну стабільність системи. Проведене тестування підтвердило повну відповідність розробленого продукту поставленим вимогам, точність статистичних розрахунків та коректність обробки вхідних даних. Таким чином, мета курсової роботи повністю досягнута, а отримані результати демонструють глибоке розуміння принципів алгоритмічного моделювання складних процесів.

ПЕРЕЛІК ПОСИЛАНЬ

1. Edelkamp S., Schroedl S. Heuristic Search: Theory and Applications. Morgan Kaufmann Publishers, 2011. 712 p.
2. Check if an instance of 8 puzzle is solvable // GeeksforGeeks. URL: <https://www.geeksforgeeks.org/check-instance-8-puzzle-solvable/>
3. Uninformed Search Algorithms in AI // Javatpoint. URL: <https://www.javatpoint.com/uninformed-search-algorithms-in-ai>
4. Windows Forms documentation // Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>
5. Asynchronous programming with async and await Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>
6. 8-puzzle // Wikipedia, The Free Encyclopedia. URL: https://en.wikipedia.org/wiki/15_puzzle

ДОДАТОК А ТЕХНІЧНЕ ЗАВДАННЯ

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

Кафедра
інформатики та програмної інженерії

Затвердив

Керівник __Куценко М.О.__

« 6 » _____ березня _____ 2026 р.

Виконавець:

Студент _____ Ярощук І.Р. _____

« 6 » _____ березня _____ 2026 р.

ТЕХНІЧНЕ ЗАВДАННЯ

на виконання курсової роботи

на тему: «Розробка інтерактивної гри "8-puzzle" з використанням алгоритмів
пошуку (BFS, LDFS, IDS)»

з дисципліни:

«Основи програмування. Курсова робота»

Київ 2026

1. *Мета:* Метою курсової роботи є набуття практичного досвіду об'єктно-орієнтованого аналізу та проєктування складних додатків, а також реалізація інтерактивної гри «8-puzzle» з порівняльним аналізом алгоритмів пошуку в ширину (BFS), лімітованого пошуку в глибину (LDFS) та пошуку з ітеративним поглибленням (IDS).

2. *Дата початку роботи:* «6» березня 2026 р.

3. *Дата закінчення роботи:* «10» травня 2026 р.

4. *Вимоги до програмного забезпечення.*

1) Функціональні вимоги:

- Можливість користувача задавати початкову розстановку цифр на полі.
- Можливість вибору користувачем одного з трьох алгоритмів розв'язання задачі: BFS, LDFS або IDS.
- Візуалізація процесу переміщення кістяшок на ігровому полі.
- Збереження отриманих результатів (шлях розв'язку, кількість кроків) у текстовий файл.
- Практична оцінка та відображення часової та просторової складності використаних алгоритмів.
- Можливість автоматичного вирішення головоломки з демонстрацією кроків.

2) Нефункціональні вимоги:

- Програмний додаток повинен мати графічний віконний інтерфейс з меню та інтерактивними елементами керування.
- Реалізація має базуватися на принципах об'єктно-орієнтованого програмування (інкапсуляція, успадкування, поліморфізм).
- Інтерфейс та реалізація кожного класу мають міститися в окремих файлах.

- Програмний код повинен бути детальним чином документований коментарями.
- Все програмне забезпечення та супроводжуюча технічна документація повинні задовольняти наступним ДЕСТам:
ДСТУ 3008 - 2015 - Розробка технічної документації.

5. Стадії та етапи розробки програмного забезпечення:

- 1) Розробка алгоритмічної складової програмного забезпечення (до 25.03.2026 р.)
- 2) Об'єктно-орієнтований аналіз предметної області завдання (до 11.03.2026 р.)
- 3) Об'єктно-орієнтоване проєктування програмного забезпечення (до 15.04.2026р.)
- 4) Розробка програмного забезпечення (до 15.04.2026р.)
- 5) Тестування розробленого програмного забезпечення (до 22.04.2026р.)
- 6) Демонстрація та захист програмного забезпечення (до 20.05.2026 р.)

6. Порядок контролю та приймання. Поточні результати роботи над КР регулярно демонструються викладачу. Своєчасність виконання основних етапів графіку підготовки роботи впливає на оцінку за КР відповідно до критеріїв оцінювання.

ДОДАТОК Б ТЕКСТИ ПРОГРАМНОГО КОДУ

*Тексти програмного коду програмного забезпечення
розв'язання головоломки "8-Puzzle" методами сліпого пошуку*

(Найменування програми (документа))

GitHub репозиторій

(Вид носія даних)

10 арк, 28,2 кб

(Обсяг програми (документа), арк.,

студента групи ІІІ-56 І курсу

Ярощука І.Р.

<https://github.com/yarxshh/EightPuzzleSolver.git>

BFSAlgorithm.cs

```
using System.Collections.Generic;

namespace EightPuzzleSolver
{
    public class BFSAlgorithm
    {
        public List<PuzzleState> FindPath(PuzzleState start, int[, ] goalBoard, out
int nodesGenerated)
        {
            nodesGenerated = 0;

            Queue<PuzzleState> queue = new Queue<PuzzleState>();

            HashSet<PuzzleState> visited = new HashSet<PuzzleState>();

            queue.Enqueue(start);
            visited.Add(start);

            while (queue.Count > 0)
            {
                PuzzleState current = queue.Dequeue();

                if (current.IsGoal(goalBoard))
                {
                    return ReconstructPath(current);
                }

                List<PuzzleState> neighbors = current.GetNeighbors();
                foreach (PuzzleState neighbor in neighbors)
                {
                    nodesGenerated++;

                    if (!visited.Contains(neighbor))
                    {
                        visited.Add(neighbor);
                        queue.Enqueue(neighbor);
                    }
                }
            }

            return null;
        }

        private List<PuzzleState> ReconstructPath(PuzzleState endState)
        {
            List<PuzzleState> path = new List<PuzzleState>();
            PuzzleState current = endState;

            while (current != null)
            {
                path.Add(current);
                current = current.Parent;
            }

            path.Reverse();
            return path;
        }
    }
}
```

IDSAlgorithm.cs

```
using System.Collections.Generic;

namespace EightPuzzleSolver
{
    public class IDSAlgorithm
    {
        public List<PuzzleState> FindPath(PuzzleState start, int[, ] goalBoard, out
int totalNodesGenerated)
        {
            totalNodesGenerated = 0;
            int maxDepth = 50;

            LDFSAlgorithm ldfs = new LDFSAlgorithm();

            for (int depthLimit = 0; depthLimit <= maxDepth; depthLimit++)
            {
                int nodesAtCurrentDepth = 0;

                List<PuzzleState> result = ldfs.FindPath(start, goalBoard,
depthLimit, out nodesAtCurrentDepth);

                totalNodesGenerated += nodesAtCurrentDepth;

                if (result != null)
                {
                    return result;
                }
            }

            return null;
        }
    }
}
```

LDFSAlgorithm.cs

```
using System.Collections.Generic;

namespace EightPuzzleSolver
{
    public class LDFSAlgorithm
    {
        public List<PuzzleState> FindPath(PuzzleState start, int[, ] goalBoard, int
depthLimit, out int nodesGenerated)
        {
            nodesGenerated = 0;

            Stack<PuzzleState> stack = new Stack<PuzzleState>();
            stack.Push(start);

            while (stack.Count > 0)
            {
                PuzzleState current = stack.Pop();

                if (current.IsGoal(goalBoard))
                {
                    return ReconstructPath(current);
                }

                if (current.Depth < depthLimit)
                {
                    List<PuzzleState> neighbors = current.GetNeighbors();
```

```

        neighbors.Reverse();

        foreach (PuzzleState neighbor in neighbors)
        {
            nodesGenerated++;

            if (!IsCycle(neighbor))
            {
                stack.Push(neighbor);
            }
        }
    }

    return null;
}

private bool IsCycle(PuzzleState state)
{
    PuzzleState ancestor = state.Parent;
    while (ancestor != null)
    {
        if (state.Equals(ancestor)) return true;
        ancestor = ancestor.Parent;
    }
    return false;
}

private List<PuzzleState> ReconstructPath(PuzzleState endState)
{
    List<PuzzleState> path = new List<PuzzleState>();
    PuzzleState current = endState;

    while (current != null)
    {
        path.Add(current);
        current = current.Parent;
    }

    path.Reverse();
    return path;
}
}

```

MainForm.cs

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace EightPuzzleSolver
{
    public partial class MainForm : Form
    {
        private readonly int[,] goalBoard = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 0 } };

        private List<PuzzleState> currentSolutionPath;

        public MainForm()
        {
            InitializeComponent();
        }
    }
}

```

```

        cmbAlgorithm.Items.Add("BFS");
        cmbAlgorithm.Items.Add("LDFS");
        cmbAlgorithm.Items.Add("IDS");
        cmbAlgorithm.SelectedIndex = 0;
        numDepthLimit.Value = 20;
    }

    private int[,] ReadBoardFromUI()
    {
        int[,] board = new int[3, 3];
        TextBox[,] grid = {
            { txt00, txt01, txt02 },
            { txt10, txt11, txt12 },
            { txt20, txt21, txt22 }
        };

        HashSet<int> usedNumbers = new HashSet<int>();

        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                string text = grid[i, j].Text.Trim();
                if (string.IsNullOrEmpty(text)) text = "0";

                if (int.TryParse(text, out int val) && val >= 0 && val <= 8)
                {
                    if (!usedNumbers.Add(val))
                        throw new Exception("Error! Numbers on the board must
not repeat.");

                    board[i, j] = val;
                }
                else
                {
                    throw new Exception("Please enter valid numbers from 0 to
8.");
                }
            }
        }
        return board;
    }

    private void UpdateGridDisplay(PuzzleState state)
    {
        TextBox[,] grid = {
            { txt00, txt01, txt02 },
            { txt10, txt11, txt12 },
            { txt20, txt21, txt22 }
        };

        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                grid[i, j].Text = state.Board[i, j] == 0 ? "" : state.Board[i,
j].ToString();
            }
        }
    }

    public void btnGenerate_Click(object sender, EventArgs e)
    {
        PuzzleGenerator generator = new PuzzleGenerator();
        PuzzleState randomState = generator.GenerateRandom();
        UpdateGridDisplay(randomState);
    }

```

```

        lblSteps.Text = "Steps: -";
        lblTime.Text = "Time: -";
        lblNodes.Text = "Nodes: -";
        currentSolutionPath = null;
    }

    public async void btnSolve_Click(object sender, EventArgs e)
    {
        try
        {
            int[,] startBoard = ReadBoardFromUI();
            PuzzleState startState = new PuzzleState(startBoard);

            PuzzleGenerator generator = new PuzzleGenerator();
            if (!generator.IsSolvable(startState.Board))
            {
                MessageBox.Show("This configuration is mathematically unsolvable.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }

            string algorithm = cmbAlgorithm.SelectedItem?.ToString() ?? "";
            int nodesGenerated = 0;
            Stopwatch stopwatch = new Stopwatch();

            btnSolve.Enabled = false;
            btnGenerate.Enabled = false;

            currentSolutionPath = await Task.Run(() =>
            {
                stopwatch.Start();
                if (algorithm.Contains("BFS"))
                    return new BFSAlgorithm().FindPath(startState, goalBoard, out nodesGenerated);
                else if (algorithm.Contains("LDFS"))
                    return new LDFSAlgorithm().FindPath(startState, goalBoard, (int)numDepthLimit.Value, out nodesGenerated);
                else if (algorithm.Contains("IDS"))
                    return new IDSAlgorithm().FindPath(startState, goalBoard, out nodesGenerated);

                return null;
            });

            stopwatch.Stop();

            if (currentSolutionPath != null)
            {
                lblSteps.Text = $"Steps: {currentSolutionPath.Count - 1}";
                lblTime.Text = $"Time: {stopwatch.ElapsedMilliseconds} ms";
                lblNodes.Text = $"Nodes: {nodesGenerated}";

                foreach (var state in currentSolutionPath)
                {
                    UpdateGridDisplay(state);
                    await Task.Delay(400);
                }
            }
            else
            {
                MessageBox.Show("Solution not found.", "Result", MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
        }
        catch (Exception ex)
        {

```

```

        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    finally
    {
        btnSolve.Enabled = true;
        btnGenerate.Enabled = true;
    }
}

public void btnSave_Click(object sender, EventArgs e)
{
    if (currentSolutionPath == null)
    {
        MessageBox.Show("No solution to save.", "Warning",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    using (SaveFileDialog sfd = new SaveFileDialog() { Filter = "Text files
    (*.txt)|*.txt", FileName = "8Puzzle_Result.txt" })
    {
        if (sfd.ShowDialog() == DialogResult.OK)
        {
            using (StreamWriter writer = new StreamWriter(sfd.FileName))
            {
                writer.WriteLine("--- 8-Puzzle Solution Results ---");
                writer.WriteLine(lblSteps.Text);
                writer.WriteLine(lblTime.Text);
                writer.WriteLine(lblNodes.Text);
                writer.WriteLine("\n--- Steps ---");

                for (int i = 0; i < currentSolutionPath.Count; i++)
                {
                    writer.WriteLine($"Step {i}:");
                    var b = currentSolutionPath[i].Board;
                    for (int r = 0; r < 3; r++)
                        writer.WriteLine($"{b[r, 0]} {b[r, 1]} {b[r, 2]}");
                    writer.WriteLine();
                }
            }
            MessageBox.Show("Successfully saved!", "Success",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
}

private void MainForm_Load(object sender, EventArgs e)
{
}
}

```

PuzzleGenerator.cs

```

using System;

namespace EightPuzzleSolver
{
    public class PuzzleGenerator
    {
        private Random random = new Random();
    }
}

```

```

public PuzzleState GenerateRandom()
{
    int[] linearBoard = { 0, 1, 2, 3, 4, 5, 6, 7, 8 };

    for (int i = linearBoard.Length - 1; i > 0; i--)
    {
        int j = random.Next(i + 1);
        int temp = linearBoard[i];
        linearBoard[i] = linearBoard[j];
        linearBoard[j] = temp;
    }

    if (!IsSolvable(linearBoard))
    {
        if (linearBoard[0] != 0 && linearBoard[1] != 0)
        {
            int temp = linearBoard[0];
            linearBoard[0] = linearBoard[1];
            linearBoard[1] = temp;
        }
        else
        {
            int temp = linearBoard[linearBoard.Length - 1];
            linearBoard[linearBoard.Length - 1] =
linearBoard[linearBoard.Length - 2];
            linearBoard[linearBoard.Length - 2] = temp;
        }
    }

    int[,] board2D = new int[3, 3];
    for (int i = 0; i < 9; i++)
    {
        board2D[i / 3, i % 3] = linearBoard[i];
    }

    return new PuzzleState(board2D);
}

public bool IsSolvable(int[] board)
{
    int inversions = 0;
    for (int i = 0; i < board.Length - 1; i++)
    {
        for (int j = i + 1; j < board.Length; j++)
        {
            if (board[i] != 0 && board[j] != 0 && board[i] > board[j])
            {
                inversions++;
            }
        }
    }
    return inversions % 2 == 0;
}

public bool IsSolvable(int[,] board)
{
    int[] linearBoard = new int[9];
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            linearBoard[i * 3 + j] = board[i, j];
        }
    }
    return IsSolvable(linearBoard);
}

```

```

    }
}

```

PuzzleState.cs

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace EightPuzzleSolver
{
    public class PuzzleState
    {
        public int[,] Board { get; set; }

        public int EmptyRow { get; set; }
        public int EmptyCol { get; set; }

        public PuzzleState Parent { get; set; }

        public int Depth { get; set; }

        public PuzzleState(int[,] board, PuzzleState parent = null, int depth = 0)
        {
            Board = (int[,])board.Clone();
            Parent = parent;
            Depth = depth;
            FindEmptyTile();
        }

        private void FindEmptyTile()
        {
            for (int i = 0; i < 3; i++)
            {
                for (int j = 0; j < 3; j++)
                {
                    if (Board[i, j] == 0)
                    {
                        EmptyRow = i;
                        EmptyCol = j;
                        return;
                    }
                }
            }
        }

        public bool IsGoal(int[,] goalBoard)
        {
            for (int i = 0; i < 3; i++)
            {
                for (int j = 0; j < 3; j++)
                {
                    if (Board[i, j] != goalBoard[i, j]) return false;
                }
            }
            return true;
        }

        public List<PuzzleState> GetNeighbors()
        {
            List<PuzzleState> neighbors = new List<PuzzleState>();
            int[] dRow = { -1, 1, 0, 0 };
            int[] dCol = { 0, 0, -1, 1 };

```

```

        for (int i = 0; i < 4; i++)
        {
            int newRow = EmptyRow + dRow[i];
            int newCol = EmptyCol + dCol[i];

            if (newRow >= 0 && newRow < 3 && newCol >= 0 && newCol < 3)
            {
                int[,] newBoard = (int[,])Board.Clone();
                newBoard[EmptyRow, EmptyCol] = newBoard[newRow, newCol];
                newBoard[newRow, newCol] = 0;

                neighbors.Add(new PuzzleState(newBoard, this, Depth + 1));
            }
        }
        return neighbors;
    }

    public override bool Equals(object obj)
    {
        if (!(obj is PuzzleState other)) return false;
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                if (Board[i, j] != other.Board[i, j]) return false;
            }
        }
        return true;
    }

    public override int GetHashCode()
    {
        int hash = 17;
        foreach (var item in Board)
        {
            hash = hash * 31 + item.GetHashCode();
        }
        return hash;
    }
}

```

Program.cs

```

namespace EightPuzzleSolver
{
    internal static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            // To customize application configuration such as set high DPI settings
            // or default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new MainForm());
        }
    }
}

```